

COMP3211/9211 Computer Architecture Assignment

Processor System with Data Preprocessing Accelerator

This assignment is a design project to be completed in groups. It is due in Week 10.

General Description

Data preprocessing is an important part in many application areas, such as machine learning and signal processing. In this project, you are required to build a processor system that consists of a CPU and a tightly coupled coprocessor with a shared data memory (DMEM), as depicted in Figure 1.

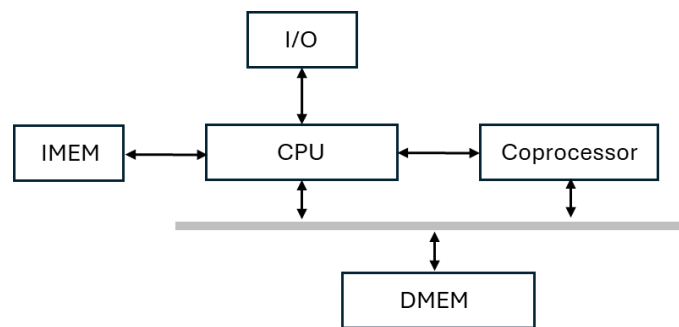


Figure 1

The CPU is a general-purpose processor. It fetches and decodes instructions from the instruction memory (IMEM). If the fetched instruction is a special instruction, the CPU will dispatch the operation of the instruction to the coprocessor; otherwise, the CPU executes the instruction itself.

The coprocessor is a FSM (Finite State Machine) that performs a set of operations commonly used for data preprocessing.

The dataset to be preprocessed is assumed to consist of vectors that are stored in IMEM, and the vector elements are represented as 8-bit values.

For a special instruction, the CPU should copy the data from IMEM to the shared memory. The coprocessor then performs the designated operation on the data and writes results back to memory upon completion.

Requirements

Your work in this project consists of ISA design, hardware design and modeling, design evaluation, and FPGA implementation, as detailed below.

1) ISA design

You need to design a simple ISA. It consists of

- i) Basic instructions to be executed by the CPU, and
- ii) Special instructions to be executed by the coprocessor.

The basic instructions should be sufficient for the CPU to perform any background tasks (such as transferring data from IMEM to DMEM, receiving control inputs, displaying execution states on the output) as well as the data preprocessing tasks not being supported by the coprocessor.

The special instructions should be cost-effective for data-preprocessing operations and scalable to different data sizes. To limit the scope of the project, we assume the coprocessor can perform only three operations on 8-bit integer vectors:

- i) vector addition
 $A = B + C$
 $A[i] = B[i] + C[i]$
- ii) vector scaling
 $A = B * k$
 $A[i] = B[i] * k$
- iii) vector clipping
 $A = \text{clip}(B)$
 $A[i] = B[i] \text{ if } B[i] \leq \text{MAX}, \text{ otherwise, } A[i] = \text{MAX}$

You are required to design special instructions that enable the execution of these operations. The instructions should specify the type of operation and may also include the locations and sizes of the input data, as well as the destination where the results should be stored.

For example, a vector addition instruction may be defined as:

VADD Rd, Rs, Rt, imme

where *VADD* denotes vector addition, *Rs* and *Rt* specify the locations of the two source vectors, *imme* specifies the vector size, and *Rd* specifies the location where the result is stored.

Depending on your ISA design, some operands may not need to be explicitly specified if they are fixed. For example, if the vector size is fixed, *imme* does not need to appear in the instruction.

2) Hardware design and modeling

You need to design a pipelined CPU for the given instruction set, a coprocessor to execute the operations specified by the special instructions, and integrate all components into a complete system, as shown in Figure 1.

To reduce energy consumption, the coprocessor is only active during the special instruction execution and is activated by the CPU.

During the special instruction execution, the CPU can either wait for the coprocessor to complete the operation or continue performing background tasks. Polling can be used to monitor the status of the coprocessor operation and determine when the operation has finished.

For simplicity, caches are not used and bus arbitration is omitted in your hardware design. Software is responsible for preventing bus contention and ensuring conflict-free access to data memory.

Once you have completed the design, build a hardware model of your solution using either VHDL or Verilog.

3) Design evaluation

To evaluate your design, you need to create a benchmark. A sample benchmark is provided at the end of this section. Your benchmark may target a different problem domain and use datasets with different structures and sizes.

For both the sample benchmark and the benchmark you create, write code for two cases:

- i) Case 1: The code does not use special instructions. It is executed entirely by the CPU.
- ii) Case 2: The code uses special instructions. It is executed cooperatively by the CPU and the coprocessor.

Use the Vivado software tool to evaluate your design: verify its functionality and analyse its performance under both cases for each benchmark.

4) Implementation

Implement your design on the lab board, using switches for input, buttons for execution control, and the LED array and 7-segment display for output.

For any information not provided in this document but needed for your design, you may make reasonable assumptions.

Sample Benchmark

Preprocess a dataset of 100 student records. Each record consists of six 8-bit fields: one student ID field and five mark fields (M1-M5).

The dataset may contain empty mark fields, and a student may appear in multiple records, as shown in Table 1. The records are already sorted by student ID.

Write a program to perform the following tasks:

- i) Replace all the empty fields with 0.
- ii) Calculate the total mark for each student by summing the five mark fields and capping the result at 100.
- iii) For records with duplicate IDs, retain the record with the highest total mark and remove all other records associated with the same ID, as illustrated in Table 2, where the fields affected by the processing are highlighted in red. Here you may assume that no ties occur; that is, for any given ID, exactly one record has the highest total mark.
- iv) Save the processed dataset in the data memory and display the average total mark of all students on the 7-segment display.

Table 1: Raw Dataset

ID	M1	M2	M3	M4	M5
10001010	15	8	9	20	45
10001100	12	9	6	15	
10001101	15	10	10	18	55
10001111	10	5	7	12	45
10010001	10	10	0	0	0
10010001	10	10	11	15	40
10010011	20	10	10	15	30
10011100	10	5	6	5	25

Table 2: Processed Dataset

ID	M1	M2	M3	M4	M5	Total
10001010	15	8	9	20	45	97
10001100	12	9	6	15	0	42
10001101	15	10	10	18	55	100
10001111	10	5	7	12	45	79
10010001	10	10	11	15	40	86
10010011	20	10	10	15	30	85
10011100	10	5	6	5	25	51

Guide to project development:

Each project group is required to make a detailed plan, including

- Task breakdown,
- Project schedule,
- Team member roles and responsibilities,
- Testing and verification methods, and
- A project management strategy.

Assessment (100 marks)

The assessment for each group consists

- group presentation (30%)
 - assessed by the tutors and peers in your TLB class in Week 10. Each group is given 25 mins for the presentation.
 - An assessment form will be available for the peer assessment.
- lab demonstration (50%)
 - assessed by the tutors in your TLB class in Week 10
- Report (20%)
 - Give submission of code and report, due **5pm** Monday, Week 11
 - zip your Vivado project and the project report. **Each group needs to submit only one copy.** The submission system will be set up on the Assignment page of the course website in Week 10.
 - Note: Report is a written form of your project design. It should be extensive in content, effective in use of figures and tables, and concise and succinct in description.

Each member needs to submit a form about the contribution of each member in their group (including themselves). On the submission form, **ensure the total contributions of all members add up to 100%.**

Your individual mark of the assignment is based on two components: your mark from the group work (G) and your relative contribution (C). It is calculated as $G \cdot C$ but capped by 100, as demonstrated in the following two examples:

1. Example 1: For a group of five members, if everyone contributes equally and the group mark is 90, each member's mark is $\min\{90 * (100\%/5) / (100\%/5), 100\} = 90$.
2. Example 2: For a group of five members with the group mark of 70, if individual contributions are 10%, 20%, 20%, 15%, and 35%, respectively, the one with the highest contribution will receive a mark of $\min\{70 * 35\% / (100\%/5), 100\} = 100$.